

计算机学院 高等程序设计 课程实验报告

实验题目：C++项目预编译对象数组指针和动态内存

学号：202500130002

日期：2026. 4. 9

班级： 25. 2

姓名： 林毅玮

Email： 18601013891@163.com

实验目的：

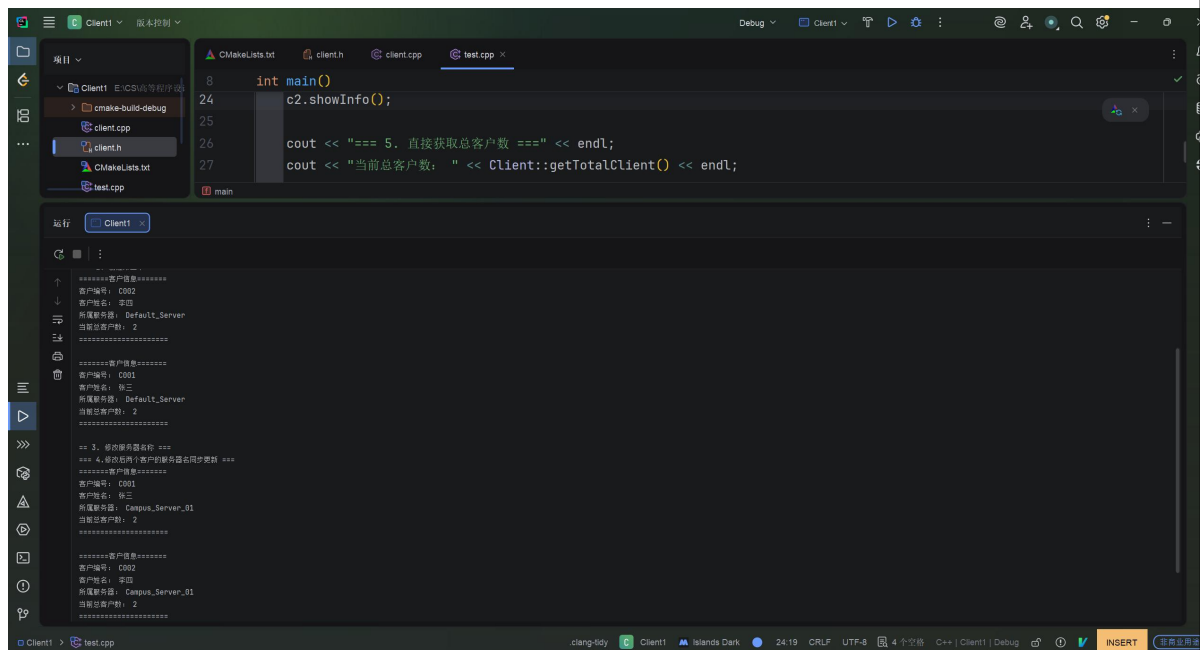
1. 会建立多文件项目，熟悉编译的预处理指令
2. 会使用对象数组编程
3. 能找出动态内存编程中易出现的错误（内存泄露）

实验软件和硬件环境： 笔记本电脑和 CLion, 小熊猫 C++

实验步骤与内容：

1. 1 多文件结构：

使用 CLion 和小熊猫 C++ 分别尝试构建 Cmake 和 Makefile 的 Client 项目，都成功，主要是拆分为头文件，类实现和主函数三个文件



```
int main()
{
    c2.showInfo();

    cout << "=== 5. 直接获取总客户数 ===" << endl;
    cout << "当前总客户数: " << Client::getTotalClient() << endl;
}
```

```
=====客户信息=====
客户编号: 002
客户姓名: 李四
所属服务器: Default_Server
当前总客户数: 2
=====
=====客户信息=====
客户编号: 001
客户姓名: 张三
所属服务器: Default_Server
当前总客户数: 2
=====
=== 3. 修改服务器名称 ===
=== 4. 修改前两个客户的服务器名称同步更新 ===
=====客户信息=====
客户编号: 001
客户姓名: 张三
所属服务器: Campus_Server_01
当前总客户数: 2
=====
=====客户信息=====
客户编号: 002
客户姓名: 李四
所属服务器: Campus_Server_01
当前总客户数: 2
=====
```

- (1) 实验体会到多文件才分可以让类声明和实现分离，代码更加清晰，便于维护
- (2) 头文件必须加上`#ifndef/#define/#endif`来防止重复包含
- (3) .h 文件被`#include` 嵌入编译，不用单独编译

2. 1 编译预处理

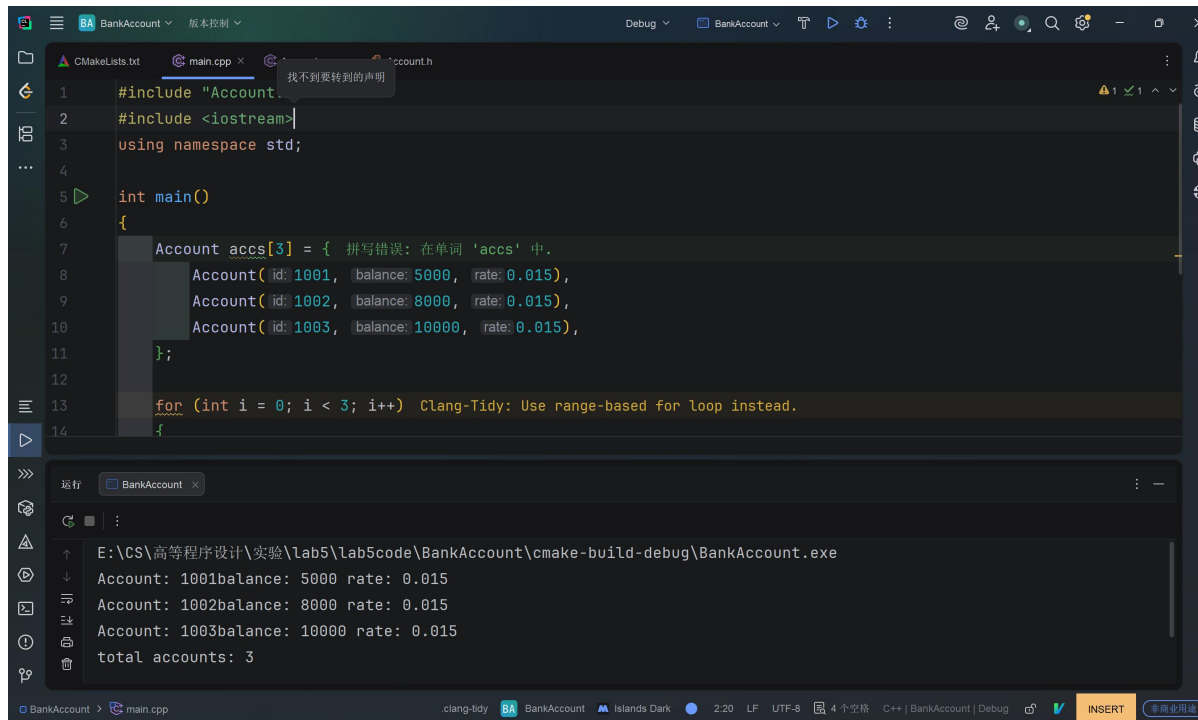
(1) 问题就是头文件被多个文件`#include`，导致重复定义，类实现与声明没有分离，函数写在了头文件

解决办法就是在所有头文件前面加上头文件保护宏，而且在头文件中只放声明

(2) 多个文件#include<iostream>但是不冲突原因:
标准库头文件<iostream>自带头文件保护, 重复包含会被忽略
#include 是文本替换, 只导入声明, 不产生重复定义的实体

(3) Dev-C++ 的标准库路径: 头文件路径是 Dev-cpp\MinGW64\include, 库文件路径是 Dev-cpp\MinGW64\lib

3 个人银行账户



```
1 #include "Account.h"
2 #include <iostream>
3 using namespace std;
4
5 int main()
6 {
7     Account accs[3] = {
8         Account(id: 1001, balance: 5000, rate: 0.015),
9         Account(id: 1002, balance: 8000, rate: 0.015),
10        Account(id: 1003, balance: 10000, rate: 0.015),
11    };
12
13    for (int i = 0; i < 3; i++)
14    {
15        Account a = accs[i];
16        cout << "Account: " << a.id << "balance: " << a.balance << "rate: " << a.rate << endl;
17    }
18    cout << "total accounts: " << accs.size() << endl;
19}
```

运行

```
E:\CS\高等程序设计\实验\lab5\lab5code\BankAccount\cmake-build-debug\BankAccount.exe
Account: 1001balance: 5000 rate: 0.015
Account: 1002balance: 8000 rate: 0.015
Account: 1003balance: 10000 rate: 0.015
total accounts: 3
```

4. 习题 cimplecircle 错误修正

常见错误:

- (1) 头文件没有保护, 重复包含报错
- (2) 构造函数没初始化成员变量
- (3) 成员函数实现未加类作用域: :
- (4) 缺少析构函数, 动态内存泄漏
- (5) 拷贝构造函数未定义, 浅拷贝问题

5. c6test.cpp 动态内存和内存泄露分析

(1) 语句的注释解析:

Object *a = new Object 动态创建对象, a 是指针

&b (*new Object (*a)) : b 是引用, 绑定到新动态创建的对象

(2) *b (new Object(*a))

引用必须初始化, 不能赋值

无*中

引用声明的时候必须绑定, 不能后续赋值

(3) 对于内存泄露解决: 所有 new 的对象必须用 delete 释放, 引用绑定的动态对象 delete

&b , 指针对象: delete a

6. 探索 Makefile 和 Cmake

- (1) Makefile 是自动化编译脚本, 按依赖关系编译多文件, 简化手动编译
- (2) CMake 是可跨平台构建工具, 生成 Makefile, 适配 Windows/Mac/Linux
- (3) 核心作用是: 管理大型 C++项目, 自动处理多文件编译和连接

结果分析和体会:

- 1. 掌握 Dev-c++多文件搭建, 头文件保护是关键
- 2. 理解编译预处理, #include 重复不冲突的原理
- 3. 会用对象数组管理多个对象
- 4. 明确 new、delete 配对, 解决内存泄漏
- 5. 理解 static、const 的成员用法与特性

就实验过程中遇到的问题及解决处理方法, 自拟 1—3 道问答题:

- 1. 为什么自定义头文件会重复包含报错, 但是<iostream>重复包含却不会?
标准头文件自带头文件保护机制#pragma once, 编译器会自动忽略重复包含, 不会报错
- 2. new 的动态对象如何避免内存泄露? new 和 delete 需要配对使用的原则, 所有动态内存分配的内存都需要手动释放, 在类中使用动态内存必须写析构函数, 在析构函数里释放内存
- 3. static 静态成员和 const 成员, 初始化方式有什么不同?
static 静态成员必须在类外和全局作用域初始化, 不能在构造函数初始化列表或者类内初始化, 而 const 成员必须在构造函数初始化列表中进行初始化, 不能在函数体内赋值